MAHARSHI OZA

Austin, TX | +1 (770) 616-5703 | maharshioza2k@gmail.com | github.com/MaharshiOzaOG

PROFESSIONAL SUMMARY

Test and validation engineer with 2 years at Magna Electronics validating ADAS camera and LiDAR-based perception: camera calibration (intrinsic/extrinsic), requirements-based acceptance criteria, recorded-data testing, and ECU bench bring-up and debug. Builds Python and C++ tools for automated testing, data analysis and pass/fail reporting, with a hardware verification background. MS in Computer Engineering, NC State.

CORE COMPETENCIES

Sensor Test & Calibration: Camera Calibration (Intrinsic/Extrinsic), Fisheye Camera Models, Planar Homography, Bird's-Eye View, Triangulation, RANSAC, Acceptance Criteria & KPI Definition, Ground-Truth Comparison
Test Automation & Data Analysis: Python (automation, report parsing, NumPy, data analysis), C++, Makefile, Tcl, Shell; automated multi-run test flows, pass/fail checks, root-cause analysis
Hardware & Systems: ECU Bench Bring-up, JTAG & GDB Debug, CAN Bus (Vector CANoe/CANalyzer), Linux Command Line, Embedded C/C++, MISRA C/C++, Scan-based Manufacturing Test (DFT)
Perception & ML: LiDAR- and Camera-based Object Detection (FOD, NFCD), Lane Detection Validation, ONNX Inference, YOLOP, PyTorch, Data Annotation, Dataset Validation, OpenCV
Engineering Practice: Git, Requirements-Based Testing (SRD/SRS), Code/Branch/Line Coverage, Regression Testing, Documentation

PROFESSIONAL EXPERIENCE

RTL Design Verification Engineer | Maxgen Technologies Pvt Ltd., India **Aug 2023 – Aug 2024**

- Implemented an out-of-order scoreboard for an AXI4 memory subsystem in SystemVerilog/UVM, tracking outstanding transactions by ID and checking read/write data, responses and per-ID ordering.
- Developed functional coverage for AXI4 bursts, IDs, out-of-order completion and handshake stalls (over 90%); verified reset, FIFO full/empty and handshake corner cases and debugged regressions in Questa, Xcelium and VCS.
- Maintained UVM environments for Asynchronous FIFO and FPU blocks and contributed to multi-port packet router RTL (FSM control, FIFOs, synchronizers).

Functions Engineer – Vision & Perception | Magna Electronics, India **Dec 2021 – Aug 2023**

- Implemented requirements-based KPI checkers in C++ for the ADAS Lane Marker Detection system, translating SRD/SRS acceptance thresholds into automated pass/fail tests for lane type, color, lateral distance to the rear axle and curvature against ground truth.
- Implemented bird's-eye-view generation in C++ from four surround-view cameras using planar homography with camera intrinsic and extrinsic calibration parameters.
- Validated the classical image-processing and DNN (ONNX) pipelines on recorded camera and CAN datasets with the same acceptance criteria, comparing detection accuracy; supported data annotation and test-set validation.
- Supported ECU bench bring-up and debug: deployed the C++ binary to target hardware, root-caused input-data failures with GDB breakpoints and a JTAG debugger, and analyzed vehicle CAN signals in Vector CANoe/CANalyzer.
- Worked on LiDAR- and camera-based Front Object Detection (FOD) and Near-Field Collision Detection (NFCD) modules.
- Implemented code/branch/line coverage and supported the 32-bit to 64-bit migration (MISRA C/C++).

PROJECTS

Surround-View Calibration, Bird's-Eye View and Lane Measurement Pipeline | C++11, Python, ONNX

- Reverse-engineered a four-camera fisheye recording format with embedded CAN data; built tools to extract frames and CAN logs.
- Implemented the fisheye camera model from calibration data (polynomial distortion, mount tilt/yaw/roll) with exact forward and inverse projection, and a lookup-table stitcher producing a 25.65 mm/px bird's-eye view.
- Wrote a dependency-free ONNX inference engine whose lane outputs match ONNX Runtime exactly, and reported per-frame lane distances from the rear axle (e.g. 1.45 m / 2.17 m, lane width ≈ 3.6 m) to CSV.

Trailer Coupler Height Estimation with a Monocular Fisheye Camera | Magna Electronics | C++, Python, OpenCV

- Undistorted fisheye frames using camera intrinsics, matched features across frames with OpenCV, estimated camera geometry with the 8-point algorithm and RANSAC, and triangulated 3D points to recover coupler height.

Automated Test Sweep and Data Analysis – 85-Run Physical Design Study | Python, Makefile, Tcl

- Automated 85 parallel tool runs with Makefiles and Python, parsing 45 metrics per run into one dataset with custom pass/fail checks, and analyzed failures to find the operating limits; the best configuration passed all checks and improved the figure of merit ~5.7×.

Scan Insertion & ATPG Setup: 8-bit Counter (Cadence DFT Fundamentals labs) | Genus, Modus, Verilog, Icarus

- Inserted mux-D scan into an 8-bit counter in Cadence Genus (45 nm GPDK): set the scan style and shift-enable, ran DFT rule checks (all 8 flops scannable) and auto-stitched one scan chain (scan_in → 8 flops → scan_out) to make every register controllable and observable.
- Generated the scan netlist, SDC, SDF and SCANDEF, plus the Modus ATPG handoff (pin-assign, test-mode and ATPG scripts).
- Verified the scan netlist at gate level with a self-checking Verilog testbench that applied load–capture–unload scan patterns, using cell models I wrote for open-source simulation; all checks passed.

UVM Verification of a Set-Associative Write-Back Cache Controller | SystemVerilog, UVM, SVA, Python

- Built a UVM 1.2 environment for a parameterized write-back/write-allocate cache controller with 3 agents (CPU, flush, reactive memory responder via TLM analysis FIFO), a reference-model scoreboard, and SVA protocol checks split into testbench-side and DUT-side assertions.

EDUCATION

M.S. Computer Engineering | North Carolina State University, Raleigh, NC | CGPA: 3.7/4 **Aug 2024 – May 2026**
Relevant Coursework: Hardware–Software Codesign, Advanced Microprocessor Architecture, ASIC Verification, Physical Design

B.Tech. Electronics & Communication Engineering | Dharmsinh Desai University, India **Aug 2018 – May 2022**
Relevant Coursework: Digital Image Processing, Digital Signal Processing, Signals and Systems, Object-Oriented Programming with C++, Microprocessors